

BÁO CÁO THỰC TẬP TUẦN 3

Chủ đề: Concurrency & Docker

Intern: Bùi Văn Nam

Team: Platform - Adtech

Gmail: buiwannam13032004@gmail.com

Leader: Nguyễn Văn Cương

1. Mục tiêu tuần

Trong tuần thứ ba, theo roadmap mục tiêu là tìm hiểu các kiến thức về lập trình đồng thời trong Java (Concurrency) và công nghệ Docker. Nội dung học tập tập trung vào việc hiểu vòng đời và cách quản lý Thread, cơ chế hoạt động của ThreadPool và ExecutorService, các phương pháp đồng bộ hóa dữ liệu trong môi trường đa luồng, cũng như cách xử lý các tác vụ bất đồng bộ bằng CompletableFuture.

Bên cạnh đó, đã tìm hiểu các khái niệm nền tảng của Docker như Container, Image, Docker Architecture, Dockerfile, Image Layer, Networking, Storage và Docker Compose. Qua đó hiểu được cách đóng gói, triển khai và quản lý ứng dụng trong môi trường Container.

Lịch học Tuần 3 (Thứ 2 - Thứ 6)

Ngày	Nội dung học	Kết quả đạt được
Thứ 2	Thread Lifecycle & Thread Management	Hiểu vòng đời của Thread, các trạng thái của Thread và cách tạo, quản lý, dừng Thread trong Java.
Thứ 3	ThreadPool, ExecutorService & Synchronization	Hiểu cơ chế ThreadPool, ExecutorService và các phương pháp đồng bộ hóa bằng synchronized, Lock và Atomic Classes.
Thứ 4	CompletableFuture & Reactive Programming Basics	Hiểu cách xử lý các tác vụ bất đồng bộ, kết hợp nhiều tác vụ và các khái niệm cơ bản của Reactive Programming.
Thứ 5	Docker Concepts, Architecture & Dockerfile	Hiểu Container, Image, Docker Architecture, Dockerfile và các nguyên tắc tối ưu Docker Image.
Thứ 6	Docker Networking, Volumes & Docker Compose	Hiểu Networking, Storage và cách sử dụng Docker Compose để triển khai ứng dụng nhiều Container.

2. Chi tiết nội dung đã học

2.1 Concurrency trong Java

a. Thread Lifecycle và Thread Management

Đã tìm hiểu về Thread và vòng đời của một Thread trong Java. Qua đó hiểu được các trạng thái khác nhau của Thread và cách quản lý Thread trong quá trình thực thi.

Lý thuyết:

Thread là một luồng thực thi trong chương trình. Một ứng dụng có thể bao gồm nhiều Thread cùng thực hiện các tác vụ khác nhau.

Vòng đời của Thread thường bao gồm các trạng thái:

- NEW: Thread được tạo nhưng chưa được bắt đầu.
- RUNNABLE: Thread đã sẵn sàng để được CPU thực thi.
- BLOCKED: Thread đang chờ lấy Lock để truy cập vào một vùng dữ liệu hoặc đoạn mã.
- WAITING: Thread đang chờ một Thread khác thực hiện một hành động.
- TIMED_WAITING: Thread tạm dừng trong một khoảng thời gian xác định.
- TERMINATED: Thread đã hoàn thành quá trình thực thi.

Một số phương thức quản lý Thread thường được sử dụng:

- start(): Bắt đầu một Thread mới.
- sleep(): Tạm dừng Thread trong một khoảng thời gian.
- join(): Chờ một Thread khác hoàn thành.
- interrupt(): Gửi tín hiệu yêu cầu Thread dừng hoặc xử lý việc bị gián đoạn.

Việc hiểu Thread Lifecycle giúp kiểm soát tốt hơn quá trình thực thi đồng thời và tránh các lỗi liên quan đến quản lý Thread.

b. ThreadPool và ExecutorService

Đã tìm hiểu về ThreadPool và ExecutorService trong Java. Qua đó hiểu được cách tái sử dụng Thread thay vì liên tục tạo mới Thread cho từng tác vụ.

Lý thuyết:

ThreadPool là một tập hợp các Thread được tạo và quản lý để thực hiện nhiều Task khác nhau. Khi một Task hoàn thành, Thread có thể được tái sử dụng cho Task tiếp theo.

Việc sử dụng ThreadPool giúp:

- Giảm chi phí tạo và hủy Thread.
- Hạn chế số lượng Thread được tạo trong hệ thống.
- Tăng hiệu năng của ứng dụng.
- Quản lý việc thực thi các Task hiệu quả hơn.

ExecutorService cung cấp các phương thức để quản lý ThreadPool:

- `execute()`: Thực thi một Task nhưng không trả về kết quả.
- `submit()`: Gửi Task và có thể nhận kết quả thông qua Future.
- `shutdown()`: Dừng việc nhận các Task mới và thực hiện quá trình tắt ThreadPool.

Một số loại ThreadPool phổ biến:

- `FixedThreadPool`: Sử dụng số lượng Thread cố định.
- `CachedThreadPool`: Tự động tạo và tái sử dụng Thread tùy theo nhu cầu.
- `ScheduledThreadPool`: Thực thi Task theo lịch hoặc sau một khoảng thời gian nhất định.

c. Synchronization và Thread Safety

Đã tìm hiểu các vấn đề phát sinh khi nhiều Thread cùng truy cập và thay đổi dữ liệu cùng chung.

Lý thuyết:

Khi nhiều Thread cùng thao tác trên một dữ liệu, có thể xảy ra Race Condition. Đây là tình trạng kết quả của chương trình phụ thuộc vào thứ tự thực thi của các Thread.

Ví dụ, khi nhiều Thread cùng tăng một biến đếm, nếu không có cơ chế đồng bộ hóa thì một số thao tác có thể bị ghi đè, dẫn đến kết quả không chính xác.

Các phương pháp đồng bộ hóa đã tìm hiểu:

- `synchronized`: Đảm bảo tại một thời điểm chỉ một Thread được truy cập vào vùng mã hoặc dữ liệu được bảo vệ.
- `Lock`: Cung cấp khả năng kiểm soát việc khóa và mở khóa linh hoạt hơn `synchronized`.
- `Atomic Classes`: Cung cấp các thao tác Thread-Safe trên dữ liệu mà không cần khóa toàn bộ vùng mã.

Một số Atomic Class phổ biến:

- `AtomicInteger`
- `AtomicLong`
- `AtomicBoolean`
- `AtomicReference`

Việc sử dụng đúng cơ chế Synchronization giúp đảm bảo tính nhất quán của dữ liệu trong môi trường đa luồng.

2.2 CompletableFuture và Reactive Programming

a. CompletableFuture

Đã tìm hiểu `CompletableFuture` và cách xử lý các tác vụ bất đồng bộ trong Java.

Lý thuyết:

CompletableFuture là một API hỗ trợ lập trình bất đồng bộ trong Java. Nó cho phép thực hiện các Task mà không cần chặn Thread hiện tại và có thể xử lý kết quả sau khi Task hoàn thành.

Một số phương thức thường sử dụng:

- `supplyAsync()`: Thực hiện một tác vụ bất đồng bộ và trả về kết quả.
- `runAsync()`: Thực hiện một tác vụ bất đồng bộ nhưng không trả về kết quả.
- `thenApply()`: Xử lý và biến đổi kết quả của Task trước đó.
- `thenAccept()`: Nhận kết quả nhưng không trả về kết quả mới.
- `thenCombine()`: Kết hợp kết quả của hai CompletableFuture.

CompletableFuture giúp xây dựng các luồng xử lý bất đồng bộ, giảm việc phải quản lý Thread thủ công và hỗ trợ kết hợp nhiều tác vụ hiệu quả.

b. Reactive Programming Basics

Đã tìm hiểu các khái niệm cơ bản của Reactive Programming và cách xử lý dữ liệu theo hướng bất đồng bộ, không chặn.

Lý thuyết:

Reactive Programming là mô hình lập trình tập trung vào việc xử lý các luồng dữ liệu và các sự kiện bất đồng bộ.

Một số đặc điểm:

- Xử lý dữ liệu theo dạng Stream.
- Hỗ trợ xử lý bất đồng bộ.
- Có khả năng xử lý nhiều sự kiện liên tục.
- Hạn chế việc Blocking Thread.

Reactive Programming thường được sử dụng trong các hệ thống có lượng lớn Request hoặc dữ liệu liên tục như API, hệ thống Streaming và các ứng dụng có yêu cầu cao về khả năng mở rộng.

2.3 Docker

a. Container Concepts và Docker Architecture

Đã tìm hiểu các khái niệm cơ bản của Docker, bao gồm Container, Image, Docker Engine và Docker Architecture.

Lý thuyết:

Docker là nền tảng giúp đóng gói ứng dụng cùng với các thư viện, Dependency và môi trường cần thiết để ứng dụng có thể chạy ổn định trên nhiều môi trường khác nhau.

Docker Image là một bản mẫu chứa các thành phần cần thiết để tạo ra Container.

Container là một Instance được tạo ra từ Docker Image và cung cấp môi trường cô lập để chạy ứng dụng.

Kiến trúc Docker bao gồm:

- Docker Client: Nhận lệnh từ người dùng.
- Docker Daemon: Quản lý Image, Container, Network và Volume.
- Docker Registry: Lưu trữ và phân phối Docker Image.

Đã tìm hiểu sự khác biệt giữa Container và Virtual Machine.

Container sử dụng chung Kernel với hệ điều hành Host nên có ưu điểm:

- Khởi động nhanh hơn.
- Sử dụng ít tài nguyên hơn.
- Kích thước nhỏ hơn.

Trong khi đó, Virtual Machine thường phải chạy một hệ điều hành Guest riêng nên tiêu tốn nhiều tài nguyên hơn.

b. Dockerfile và Best Practices

Đã tìm hiểu Dockerfile và cách sử dụng Dockerfile để xây dựng Docker Image.

Lý thuyết:

Dockerfile là một file chứa các hướng dẫn để Docker xây dựng Image.

Một số Instruction thường sử dụng:

- FROM: Chọn Base Image.
- WORKDIR: Thiết lập thư mục làm việc.
- COPY: Sao chép file vào Image.
- RUN: Thực thi lệnh trong quá trình Build Image.
- CMD: Xác định lệnh mặc định khi Container chạy.
- EXPOSE: Khai báo Port mà ứng dụng sử dụng.

Một số Best Practices:

- Sử dụng Base Image có kích thước nhỏ khi phù hợp.
- Sử dụng file .dockerignore để loại bỏ các file không cần thiết.
- Sắp xếp các Instruction hợp lý để tận dụng Docker Layer Cache.
- Sử dụng Multi-stage Build để giảm kích thước Docker Image.
- Chỉ cài đặt các Dependency thực sự cần thiết.

Việc tối ưu Dockerfile giúp giảm thời gian Build và giảm kích thước Docker Image.

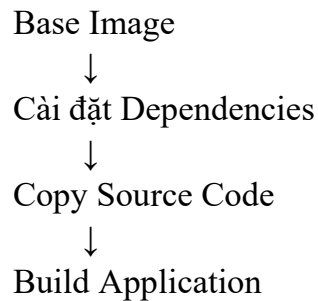
c. Image Layers và Optimization

Đã tìm hiểu cách Docker Image được xây dựng từ nhiều Layer.

Lý thuyết:

Docker Image được cấu tạo từ nhiều Layer khác nhau. Mỗi Instruction trong Dockerfile có thể tạo ra một Layer mới.

Ví dụ:



Docker sử dụng Layer Cache để tái sử dụng các Layer không thay đổi trong quá trình Build. Điều này giúp giảm thời gian Build Image.

Việc sắp xếp Dockerfile hợp lý giúp tận dụng Layer Cache hiệu quả. Ví dụ, các file chứa Dependency nên được Copy và cài đặt trước Source Code để khi Source Code thay đổi, các Layer Dependency vẫn có thể được sử dụng lại.

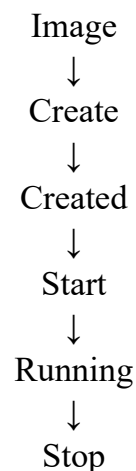
Ngoài ra, Multi-stage Build cho phép tách quá trình Build Application khỏi môi trường Runtime, giúp tạo ra Docker Image cuối cùng có kích thước nhỏ hơn.

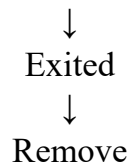
d. Container Lifecycle

Đã tìm hiểu vòng đời của Container từ khi được tạo đến khi bị xóa.

Lý thuyết:

Vòng đời cơ bản của Container:





Một số lệnh thường sử dụng:

- docker create: Tạo Container nhưng chưa chạy.
- docker start: Khởi động Container.
- docker stop: Dừng Container.
- docker restart: Khởi động lại Container.
- docker rm: Xóa Container.

Lệnh docker run thường kết hợp quá trình tạo và khởi động Container.

2.4 Docker Networking và Storage

a. Container Networking

Đã tìm hiểu các loại Network phổ biến trong Docker và cách các Container giao tiếp với nhau.

Lý thuyết:

Docker Network cho phép các Container giao tiếp với nhau hoặc giao tiếp với bên ngoài.

Một số loại Network phổ biến:

- Bridge Network: Network mặc định và thường được sử dụng để kết nối các Container trên cùng một Docker Host.
- Host Network: Container sử dụng trực tiếp Network của Host.
- Overlay Network: Cho phép kết nối các Container trên nhiều Docker Host khác nhau.

Trong Docker Compose, các Service thường có thể giao tiếp với nhau thông qua tên Service thay vì sử dụng localhost.

Ví dụ, nếu một ứng dụng Backend cần kết nối đến Database có tên Service là mysql thì có thể sử dụng:

mysql:3306

Thay vì:

localhost:3306

Điều này giúp các Container có thể giao tiếp với nhau trong cùng một Docker Network.

b. Docker Storage

Đã tìm hiểu các phương pháp lưu trữ dữ liệu trong Docker.

Lý thuyết:

Dữ liệu được lưu trực tiếp bên trong Container có thể bị mất khi Container bị xóa. Vì vậy, Docker cung cấp các cơ chế Storage để lưu trữ dữ liệu lâu dài.

Các loại Storage phổ biến:

- Bind Mount: Liên kết trực tiếp một thư mục hoặc file trên Host với thư mục trong Container.
- Docker Volume: Do Docker quản lý và thường được sử dụng để lưu trữ dữ liệu Persistent như Database.
- Tmpfs: Lưu dữ liệu trong bộ nhớ RAM, có tốc độ cao nhưng dữ liệu sẽ mất khi Container dừng.

Docker Volume thường được sử dụng cho các hệ thống Database vì giúp dữ liệu tồn tại độc lập với vòng đời của Container.

2.5 Docker Compose

Đã tìm hiểu Docker Compose và cách sử dụng để quản lý các ứng dụng gồm nhiều Container.

Lý thuyết:

Docker Compose cho phép định nghĩa nhiều Service trong một file cấu hình thường có tên là docker-compose.yml.

Một ứng dụng có thể bao gồm:

- Backend Application.
- Database.
- Redis.
- Message Queue.

Thay vì phải chạy từng Container riêng lẻ, Docker Compose cho phép quản lý toàn bộ hệ thống bằng các lệnh đơn giản.

Một số lệnh phổ biến:

- docker compose up: Khởi động các Service.
- docker compose down: Dừng và xóa các Container được tạo bởi Compose.
- docker compose build: Build các Docker Image.
- docker compose logs: Xem Log của các Service.

Docker Compose giúp đơn giản hóa việc triển khai và quản lý các ứng dụng Multi-container.

3. Kết quả đạt được

Sau khi hoàn thành tuần học thứ ba, đã đạt được các kết quả sau:

- Hiểu khái niệm Concurrency và vai trò của Thread trong Java.
- Nắm được vòng đời của Thread và các phương thức quản lý Thread.
- Hiểu cơ chế hoạt động và lợi ích của ThreadPool.
- Biết cách sử dụng ExecutorService để quản lý và thực thi các Task.
- Hiểu các vấn đề Race Condition và Thread Safety trong môi trường đa luồng.
- Nắm được các phương pháp Synchronization bằng synchronized, Lock và Atomic Classes.
- Hiểu cách sử dụng CompletableFuture để xử lý các tác vụ bất đồng bộ.
- Nắm được các khái niệm cơ bản của Reactive Programming.
- Hiểu sự khác biệt giữa Docker Container và Virtual Machine.
- Nắm được Docker Architecture và mối quan hệ giữa Docker Image và Container.
- Hiểu cách xây dựng Docker Image bằng Dockerfile.
- Nắm được cơ chế Image Layer và Layer Caching để tối ưu quá trình Build.
- Hiểu vòng đời của Container từ Create, Start, Stop đến Remove.
- Nắm được các loại Docker Network như Bridge, Host và Overlay.
- Hiểu sự khác biệt giữa Bind Mount, Docker Volume và Tmpfs.
- Biết cách sử dụng Docker Compose để quản lý các ứng dụng Multi-container.
- Xây dựng được nền tảng về lập trình đồng thời và triển khai ứng dụng bằng Container, tạo tiền đề để tiếp tục học các kiến thức về Backend Development, Spring Boot, Microservices và Cloud Deployment.

4. Kế hoạch tuần 4

Tuần 4: Database, Logging & I/O

Yêu cầu Database:

- MySQL/PostgreSQL installation và configuration
- SQL optimization với EXPLAIN plans
- Database normalization và indexing strategies
- ACID properties và transaction isolation levels

Yêu cầu Logging:

- Logging frameworks: SLF4J, Logback, Log4j2
- Log levels và appropriate usage
- Structured logging (JSON format)
- Log aggregation và centralized logging
- Performance impact của logging

Yêu cầu I/O:

- Blocking I/O vs Non-blocking I/O concepts
- Java NIO.2 (AsynchronousFileChannel, AsynchronousSocketChannel)
- Event-driven file processing
- High-performance I/O patterns
- Memory-mapped files và zero-copy techniques

Lý thuyết Logging:

- **Log Levels:** TRACE, DEBUG, INFO, WARN, ERROR
- **Appenders:** Console, File, Rolling File, Async
- **Log Patterns:** Timestamp, thread, class, message formatting
- **Structured Logging:** Key-value pairs, JSON format

Lý thuyết Event Loop & Async I/O:

- **Event Loop Implementation:** Single-threaded event processing
- **Callback Hell:** Problem và solutions (Promises, CompletableFuture)
- **Reactive Streams:** Backpressure handling
- **NIO Selector:** Multiplexing multiple channels
- **Async File I/O:** Non-blocking file operations
- **Zero-Copy:** Direct memory access, sendfile() system call